Multi-Day Water Tank Simulation

Problem statement

Simulate changing water inflow and outflow, update tank levels, detect overflow and shortages, and find the first interval in which the water level reaches a target threshold.

Assumptions and rules

• The tank has a fixed capacity and an initial water level.

• Each day's inflow and requested outflow are non-negative quantities.

• Outflow is limited to the water currently available; an unmet outflow is recorded as shortage.

• Water above capacity is recorded as overflow and discarded.

• The target threshold is checked after each day's inflow and outflow.

Algorithm

1. Set capacity, initial level, and target threshold.

2. Read daily inflow and outflow pairs.

3. For each day, add inflow to current level.

4. If level exceeds capacity, record and remove overflow.

5. If requested outflow exceeds available water, record shortage and set level to zero.

6. Otherwise subtract outflow.

7. Check whether the target level has been reached for the first time.

8. Print daily details and final status.

Pseudocode

```
START\nINPUT capacity, initial_level, target, daily_changes\nlevel = initial_level; first_day = 0\nFOR each day, inflow, ou
```

Flowchart

```
[Start] -> [Set capacity, level, target]\n   |\n[Read next day's inflow/outflow]\n   |\n[Add inflow]\n   |\n{Above capacity
```

Python solution

The program uses beginner-level Python constructs from the assigned syllabus: variables, data types, operators, conditions, loops, strings, lists, tuples, and unpacking where appropriate.

```
capacity = 1000
level = 400
target = 700
daily_changes = [
    (500, 200),
    (300, 100),
    (100, 250),
    (0, 900),
    (600, 100)
]

first_target_day = 0
total_overflow = 0
total_shortage = 0

for day in range(len(daily_changes)):
    inflow, requested_outflow = daily_changes[day]
    level = level + inflow
    overflow = 0
    shortage = 0
```

```
    if level > capacity:
        overflow = level - capacity
        total_overflow = total_overflow + overflow
        level = capacity

    if requested_outflow > level:
        shortage = requested_outflow - level
        total_shortage = total_shortage + shortage
        level = 0
    else:
        level = level - requested_outflow

    if level >= target and first_target_day == 0:
        first_target_day = day + 1

    print("Day", day + 1, "Level:", level,
          "Overflow:", overflow, "Shortage:", shortage)

print("First day target reached:", first_target_day)
print("Total overflow:", total_overflow)
print("Total shortage:", total_shortage)
print("Final water level:", level)
```

Sample output

```
Day 1 Level: 700 Overflow: 0 Shortage: 0\nDay 2 Level: 900 Overflow: 0 Shortage: 0\nDay 3 Level: 750 Overflow: 0 Shortage: 
```

Explanation / observations

The target is checked at the end of each interval. Shortage means requested outflow that could not be supplied from the tank.

Conclusion

This solution models the stated case using a clear sequence of input, processing, decisions, repetition, and output. The assumptions and sample data can be changed to test other cases.